A-02. 프로세스 정의서 — VOD 서비스 (To-Be Process)

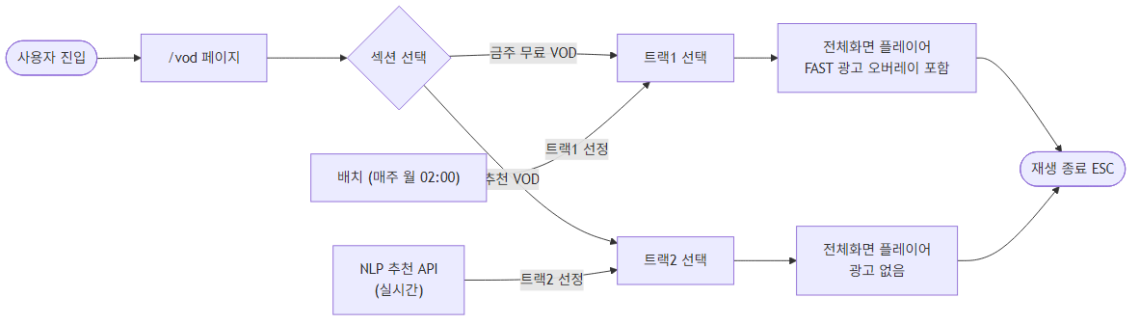
> ***문서 정보***

항목	내용
프로젝트명	2026_TV — VOD 서비스
문서 번호	A-02 (VOD)
문서 버전	v1.0
작성일	2026-03-04
범위	**VOD 탐색·재생 / NLP 추천 / FAST 광고 배치 / TF-IDF 모델 갱신**

---

1. VOD 서비스 전체 흐름

```
graph LR
    START([사용자 진입]) --> VOD["/vod 페이지"]
    VOD --> SEC{섹션 선택}
    SEC --> |금주 무료 VOD| T1["트랙1 선택"]
    SEC --> |추천 VOD| T2["트랙2 선택"]
    T1 --> PLAY1["전체화면 플레이어<br/>FAST 광고 오버레이 포함"]
    T2 --> PLAY2["전체화면 플레이어<br/>광고 없음"]
    PLAY1 --> END([재생 종료 ESC])
    PLAY2 --> END

    BATCH["배치 (매주 월 02:00)"] --> |트랙1 선정| T1
    NLP["NLP 추천 API<br/>(실시간)"] --> |트랙2 선정| T2
```

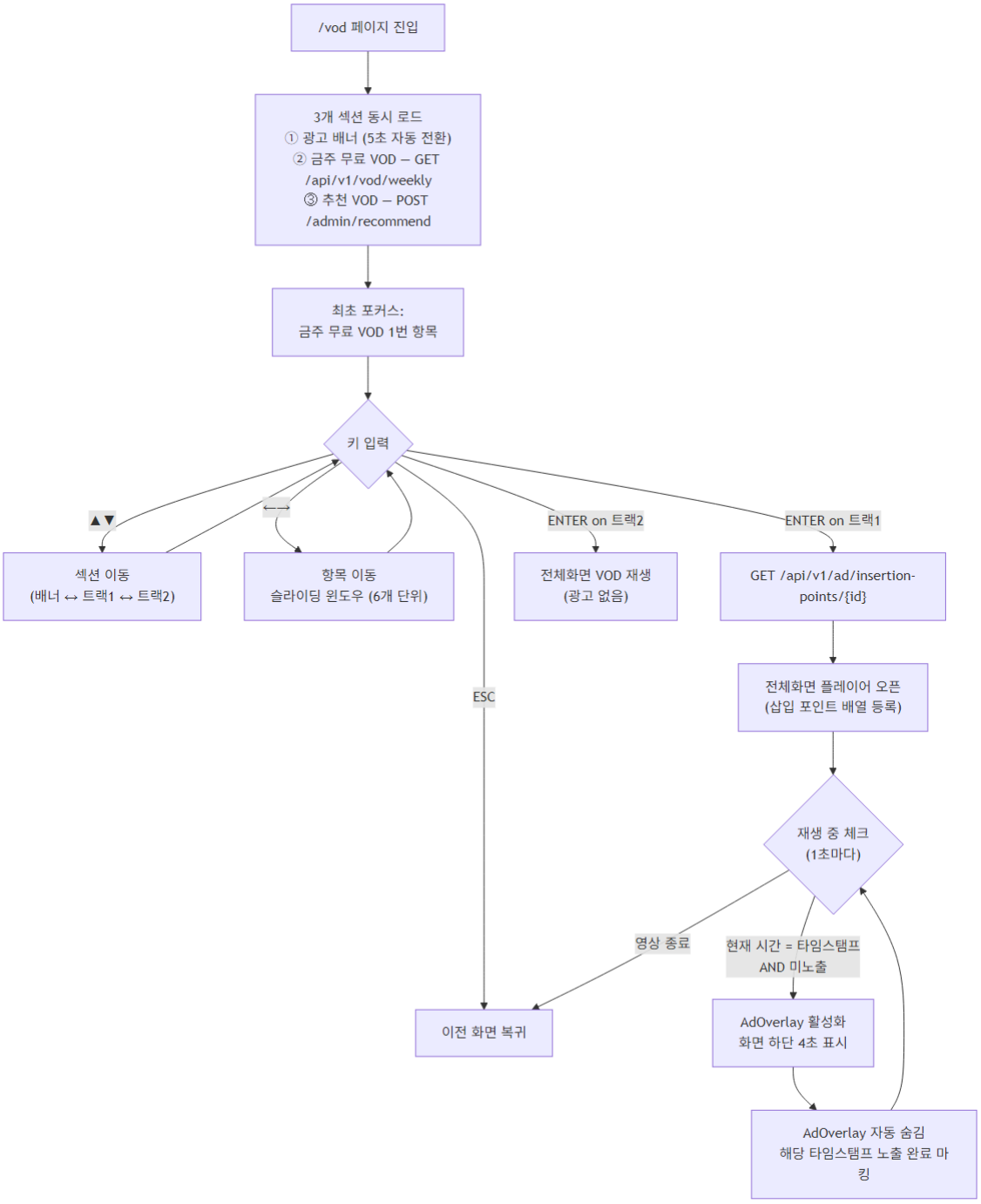


---

2. VOD 탐색 및 재생 프로세스 (P-VOD-01)

```
flowchart TD
    VOD_PAGE["/vod 페이지 진입"] --> LOAD_ALL["3개 섹션 동시 로드<br/>① 광고 배너 (5초 자동 전환)
<br/>② 금주 무료 VOD - GET /api/v1/vod/weekly<br/>③ 추천 VOD - POST /admin/recommend"]
    LOAD_ALL --> FOCUS["최초 포커스:<br/>금주 무료 VOD 1번 항목"]
```

```
    FOCUS --> KEY_NAV{키 입력}
    KEY_NAV --> |"▲▼"| SECTION_MOVE["섹션 이동<br/>(배너 ↔ 트랙1 ↔ 트랙2)"]
    KEY_NAV --> |"←→"| ITEM_MOVE["항목 이동<br/>슬라이딩 윈도우 (6개 단위)"]
    KEY_NAV --> |"ENTER on 트랙1"| REQ_AD["GET /api/v1/ad/insertion-points/{id}"]
    KEY_NAV --> |"ENTER on 트랙2"| PLAY_FREE["전체화면 VOD 재생<br/>(광고 없음)"]
    KEY_NAV --> |"ESC"| BACK["이전 화면 복귀"]
    SECTION_MOVE --> KEY_NAV
    ITEM_MOVE --> KEY_NAV

    REQ_AD --> PLAYER["전체화면 플레이어 오픈<br/>(삽입 포인트 배열 등록)"]
    PLAYER --> AD_LOOP{"재생 중 체크<br/>(1초마다)"}
    AD_LOOP --> |"현재 시간 = 타임스탬프<br/>AND 미노출"| OVERLAY["AdOverlay 활성화<br/>화면 하단 4초 표시"]
    OVERLAY --> HIDE["AdOverlay 자동 숨김<br/>해당 타임스탬프 노출 완료 마킹"]
    HIDE --> AD_LOOP
    AD_LOOP --> |"영상 종료"| BACK
```

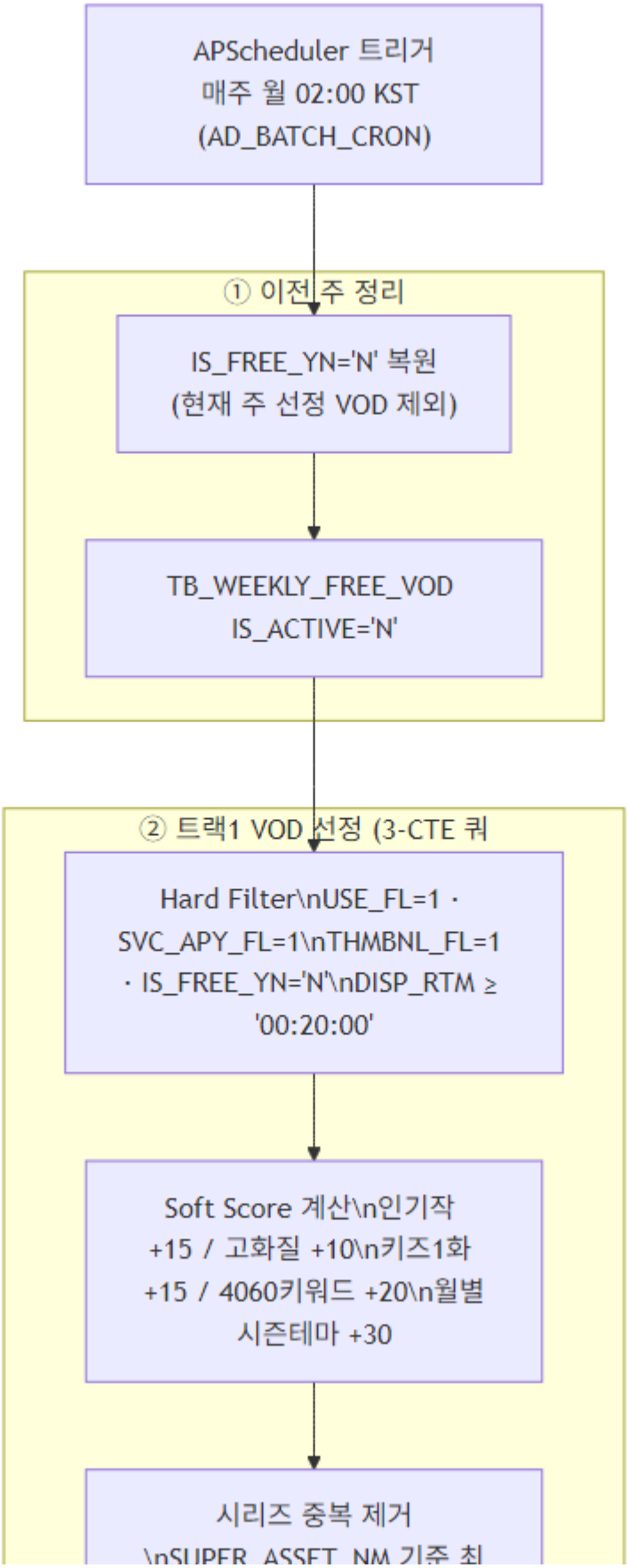


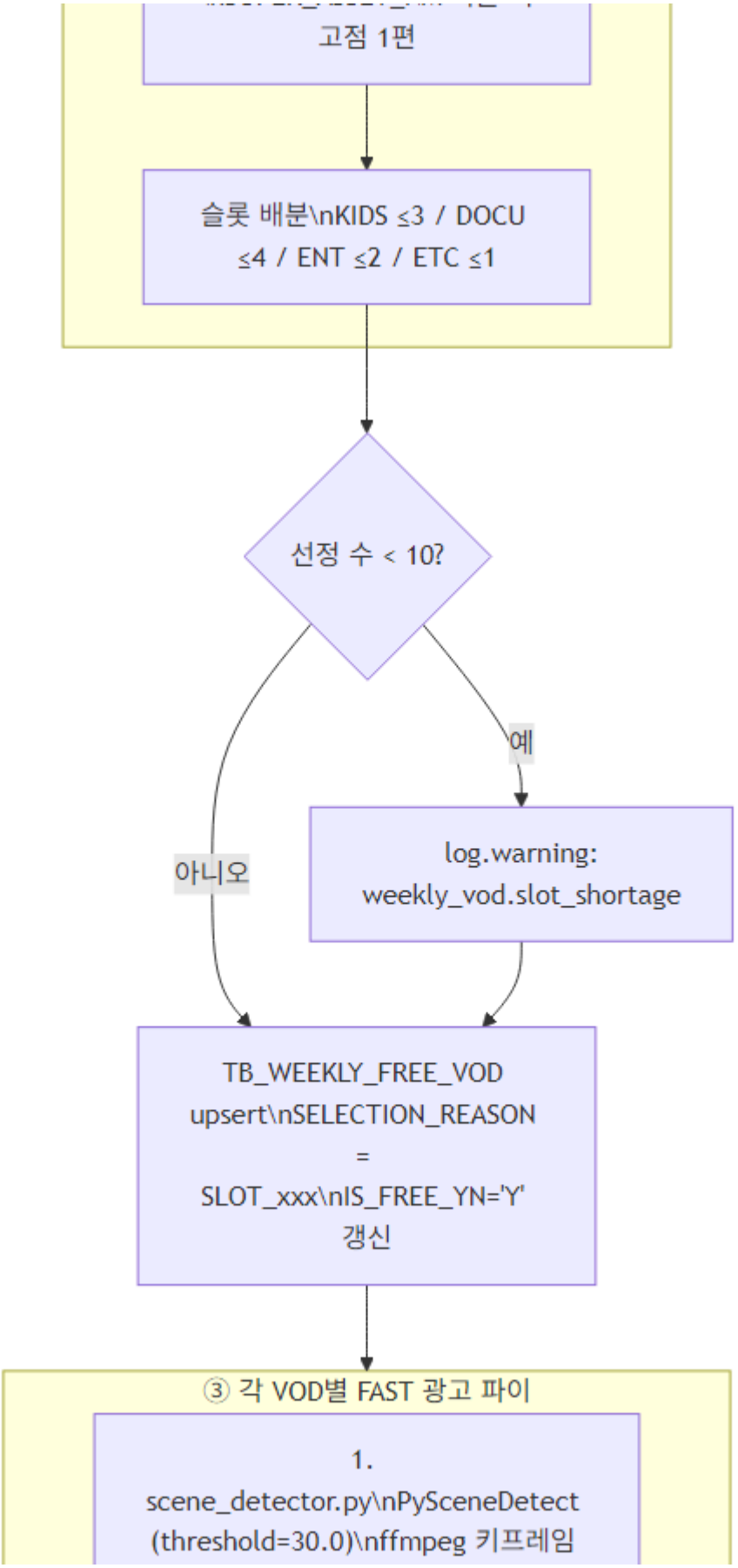
---

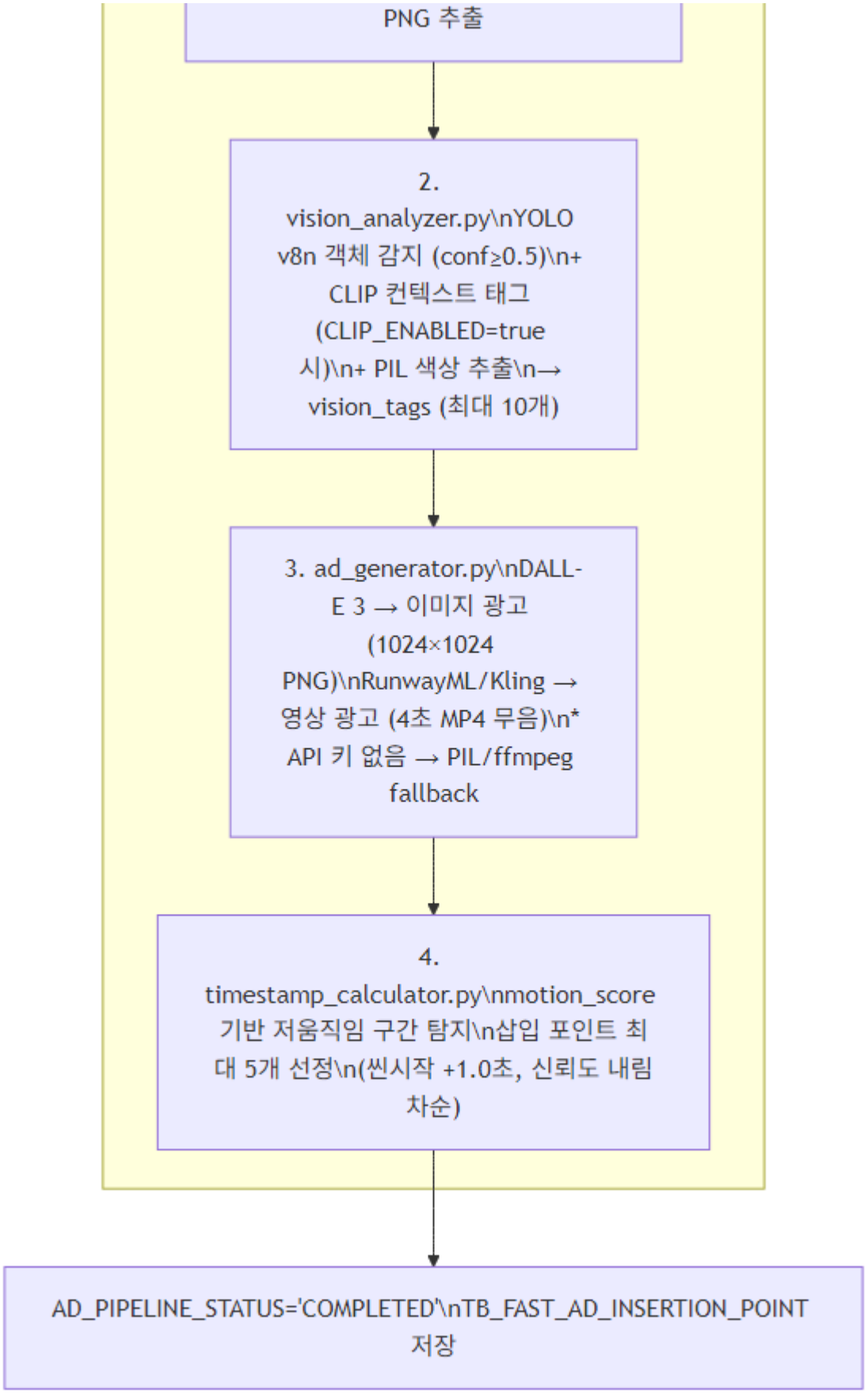
3. FAST 광고 배치 프로세스 (P-VOD-02) — 매주 월요일 02:00

```
flowchart TD
    TRIGGER["APScheduler 트리거<br/>매주 월 02:00 KST (AD_BATCH_CRON)"] --> RESTORE

    subgraph STEP0 ["① 이전 주 정리"]
        RESTORE["IS_FREE_YN='N' 복원<br/>(현재 주 선정 VOD 제외)"] --> DEACTIVATE
```

```
        DEACTIVATE["TB_WEEKLY_FREE_VOD IS_ACTIVE='N'"]
    end

    DEACTIVATE --> STEP1_START

    subgraph STEP1 ["② 트랙1 VOD 선정 (3-CTE 쿼리)"]
        STEP1_START["Hard FilterWnUSE_FL=1 · SVC_APY_FL=1WnTHMBNL_FL=1 · IS_FREE_YN='N'WnDISP_RTM
≥ '00:20:00'"] --> SOFT_SCORE
        SOFT_SCORE["Soft Score 계산Wn인기작 +15 / 고화질 +10Wn키즈1화 +15 / 4060키워드 +20Wn월별시즌
테마 +30"] --> DEDUP
        DEDUP["시리즈 중복 제거WnSUPER_ASSET_NM 기준 최고점 1편"] --> SLOT
        SLOT["슬롯 배분WnKIDS ≤3 / DOCU ≤4 / ENT ≤2 / ETC ≤1"]
    end

    SLOT --> CHECK{"선정 수 < 10?"}
    CHECK --> |"예"| WARN["log.warning:<br/>weekly_vod.slot_shortage"]
    CHECK --> |"아니오"| UPSERT
    WARN --> UPSERT

    UPSERT["TB_WEEKLY_FREE_VOD upsertWnSELECTION_REASON = SLOT_xxxWnIS_FREE_YN='Y' 갱신"] -->
PIPELINE_LOOP

    subgraph PIPELINE_LOOP ["③ 각 VOD별 FAST 광고 파이프라인 (4단계)"]
        S1["1. scene_detector.pyWnPySceneDetect (threshold=30.0)Wnffmpeg 키프레임 PNG 추출"] --> S2
        S2["2. vision_analyzer.pyWnYOLO v8n 객체 감지 (conf≥0.5)Wn+ CLIP 컨텍스트 태그
(CLIP_ENABLED=true 시)Wn+ PIL 색상 추출Wn→ vision_tags (최대 10개)"] --> S3
        S3["3. ad_generator.pyWnDALL-E 3 → 이미지 광고 (1024×1024 PNG)WnRunwayML/Kling → 영상 광고
(4초 MP4 무음)Wn* API 키 없음 → PIL/ffmpeg fallback"] --> S4
        S4["4. timestamp_calculator.pyWnmotion_score 기반 저움직임 구간 탐지Wn삽입 포인트 최대 5개 선
정Wn(씬시작 +1.0초, 신뢰도 내림차순)"]
    end

    S4 --> DONE["AD_PIPELINE_STATUS='COMPLETED'WnTB_FAST_AD_INSERTION_POINT 저장"]
```

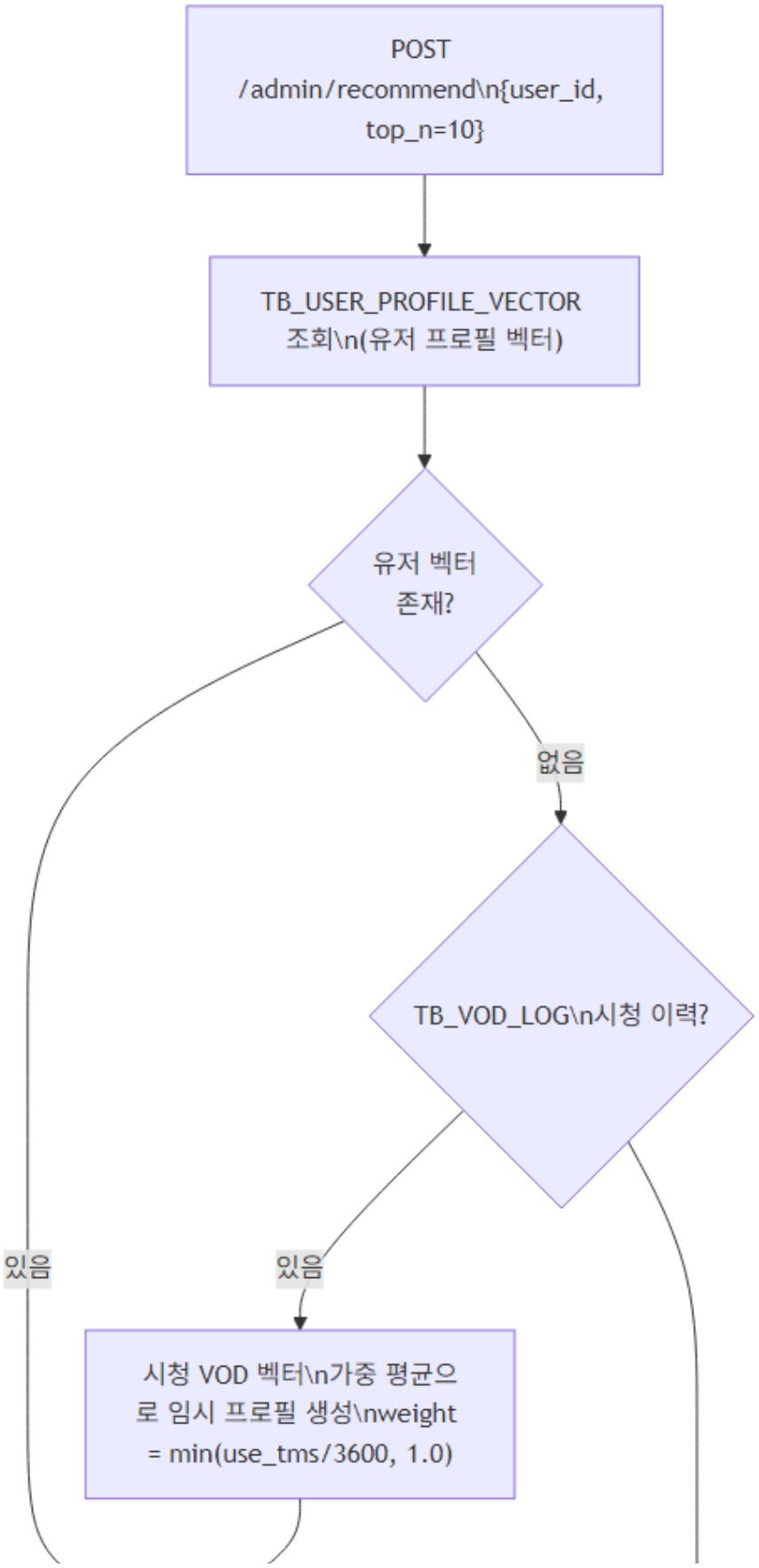

APScheduler 트리거
매주 월 02:00 KST
(AD_BATCH_CRON)
① 이전 주 정리
IS_FREE_YN='N' 복원
(현재 주 선정 VOD 제외)
TB_WEEKLY_FREE_VOD
IS_ACTIVE='N'
② 트랙1 VOD 선정 (3-CTE 쿼
Hard Filter\nUSE_FL=1 · SVC_APY_FL=1\nTHMBNL_FL=1 · IS_FREE_YN='N'\nDISP_RTM ≥ '00:20:00'
Soft Score 계산\n인기작 +15 / 고화질 +10\n키즈1화 +15 / 4060키워드 +20\n월별 시즌테마 +30
시리즈 중복 제거
\nSUPER_ASSET_NM 기준 최

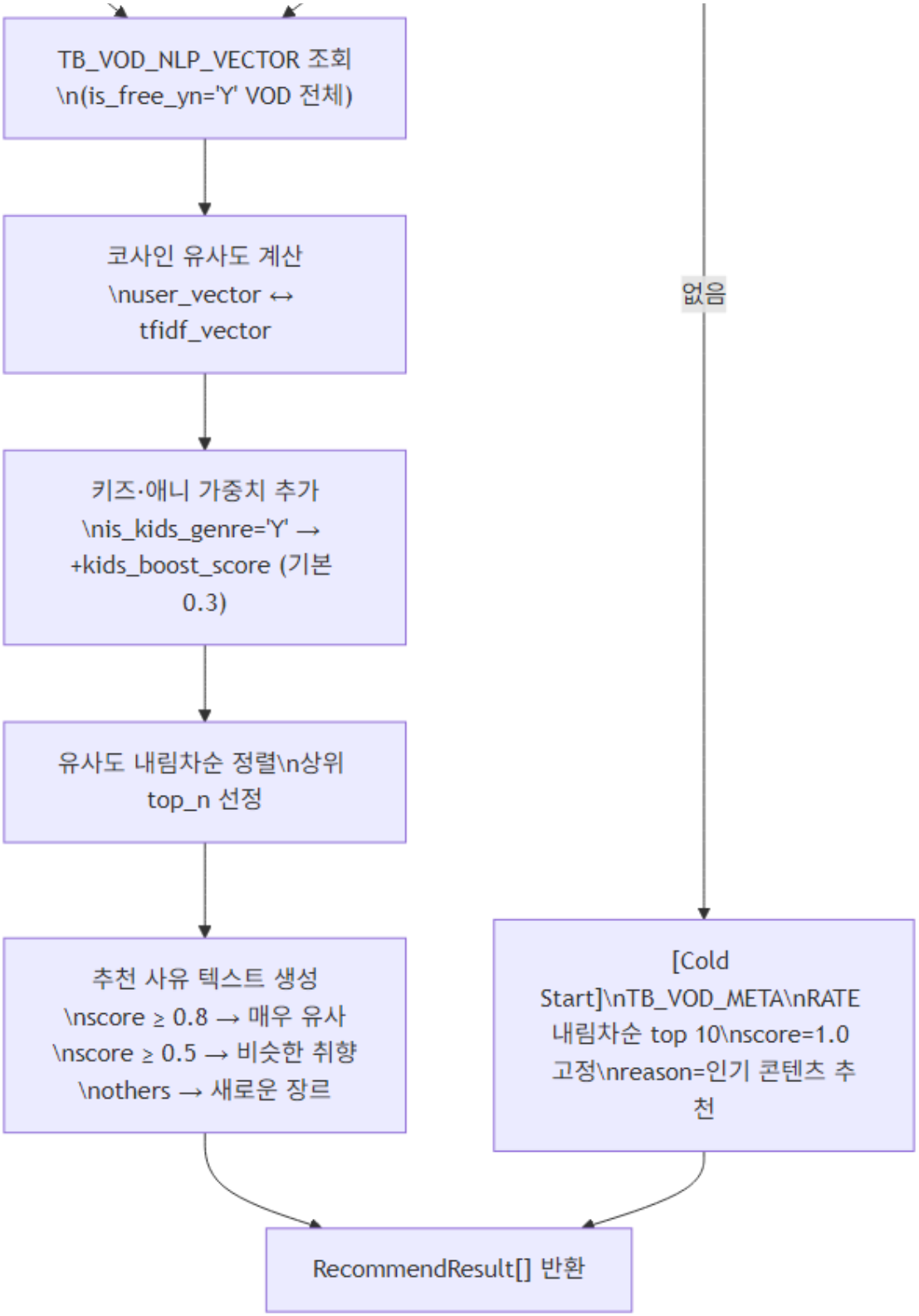

고정 1편
슬롯 배분\nKIDS ≤3 / DOCU ≤4 / ENT ≤2 / ETC ≤1
선정 수 < 10?
예
아니오
log.warning: weekly_vod.slot_shortage
TB_WEEKLY_FREE_VOD upsert\nSELECTION_REASON = SLOT_xxx\nIS_FREE_YN='Y' 갱신
③ 각 VOD별 FAST 광고 파이
1. scene_detector.py\nPySceneDetect (threshold=30.0)\nffmpeg 키프레임


PNG 추출
2. vision_analyzer.py\nYOLO v8n 객체 감지 (conf≥0.5)\n+ CLIP 컨텍스트 태그 (CLIP_ENABLED=true 시)\n+ PIL 색상 추출\n→ vision_tags (최대 10개)
3. ad_generator.py\nDALL-E 3 → 이미지 광고 (1024×1024 PNG)\nRunwayML/Kling → 영상 광고 (4초 MP4 무음)\n* API 키 없음 → PIL/ffmpeg fallback
4. timestamp_calculator.py\nmotion_score 기반 저움직임 구간 탐지\n삽입 포인트 최대 5개 선정\n(씬시작 +1.0초, 신뢰도 내림차순)
AD_PIPELINE_STATUS='COMPLETED'\nTB_FAST_AD_INSERTION_POINT 저장

4. NLP 개인화 추천 프로세스 (P-VOD-03)

```
flowchart TD
    REQ["POST /admin/recommend₩n{user_id, top_n=10}"] --> PROFILE["TB_USER_PROFILE_VECTOR 조회₩n(유저 프로필 벡터)"]

    PROFILE --> EXIST{유저 벡터<br/>존재?}

    EXIST --> |있음| VECS["TB_VOD_NLP_VECTOR 조회₩n(is_free_yn='Y' VOD 전체)"]

    EXIST --> |없음| LOG_CHECK{"TB_VOD_LOG₩n시청 이력?"}
    LOG_CHECK --> |있음| TEMP["시청 VOD 벡터₩n가중 평균으로 임시 프로필 생성₩nweight = min(use_tms/3600, 1.0)"]
    LOG_CHECK --> |없음| COLD["[Cold Start]₩nTB_VOD_META₩nRATE 내림차순 top 10₩nscore=1.0 고정₩nreason=인기 콘텐츠 추천"]
    TEMP --> VECS

    VECS --> COSINE["코사인 유사도 계산₩nuser_vector ↔ tfidf_vector"]
    COSINE --> BOOST["키즈·애니 가중치 추가₩nis_kids_genre='Y' → +kids_boost_score (기본 0.3)"]
    BOOST --> RANK["유사도 내림차순 정렬₩n상위 top_n 선정"]
    RANK --> REASON["추천 사유 텍스트 생성₩nscore ≥ 0.8 → 매우 유사₩nscore ≥ 0.5 → 비슷한 취향₩nothers → 새로운 장르"]
    REASON --> RESULT["RecommendResult[] 반환"]
    COLD --> RESULT
```


POST /admin/recommend\n{user_id, top_n=10}
TB_USER_PROFILE_VECTOR 조회\n(유저 프로필 벡터)
유저 벡터 존재?
없음
TB_VOD_LOG\n시청 이력?
있음
있음
시청 VOD 벡터\n가중 평균으로 임시 프로필 생성\nweight = min(use_tms/3600, 1.0)

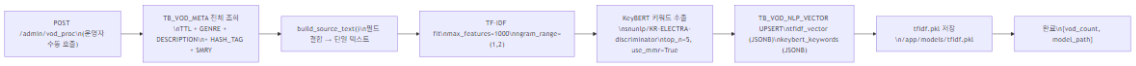


---

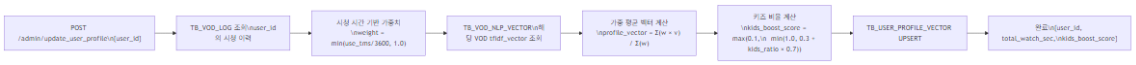
5. TF-IDF 모델 갱신 프로세스 (P-VOD-04) — 관리자 수동 실행

```
flowchart LR
    ADMIN["POST /admin/vod_proc₩n(운영자 수동 호출)"] --> META["TB_VOD_META 전체 조회₩nTTL + GENRE +
DESCRIPTION₩n+ HASH_TAG + SMRY"]
    META --> BUILD["build_source_text()₩n필드 결합 → 단일 텍스트"]
    BUILD --> FIT["TF-IDF fit₩nmax_features=1000₩nngram_range=(1,2)"]
    FIT --> KEYBERT["KeyBERT 키워드 추출₩nsnunlp/KR-ELECTRA-discriminator₩ntop_n=5, use_mmr=True"]
```

```
    KEYBERT --> UPSERT["TB_VOD_NLP_VECTOR UPSERT₩ntfidf_vector (JSONB)₩nkeybert_keywords (JSONB)"]
    UPSERT --> PICKLE["tfidf.pkl 저장₩n/app/models/tfidf.pkl"]
    PICKLE --> DONE["완료₩n{vod_count, model_path}"]
```



---

6. 유저 프로필 벡터 갱신 프로세스 (P-VOD-05)

```
flowchart LR
    TRIGGER2["POST /admin/update_user_profile₩n{user_id}"] --> LOG["TB_VOD_LOG 조회₩nuser_id의 시청 이력"]
    LOG --> WEIGHT["시청 시간 기반 가중치₩nweight = min(use_tms/3600, 1.0)"]
    WEIGHT --> NLP_VECS["TB_VOD_NLP_VECTOR₩n해당 VOD tfidf_vector 조회"]
    NLP_VECS --> PROFILE_CALC["가중 평균 벡터 계산₩nprofile_vector = Σ(w × v) / Σ(w)"]
    PROFILE_CALC --> KIDS_CALC["키즈 비율 계산₩nkids_boost_score = max(0.1,₩n  min(1.0, 0.3 + kids_ratio × 0.7))"]
    KIDS_CALC --> SAVE["TB_USER_PROFILE_VECTOR UPSERT"]
    SAVE --> DONE2["완료₩n{user_id, total_watch_sec,₩nkids_boost_score}"]
```